

Loyola Marymount University

CMSI 3300 Artificial Intelligence

Fall 2024

Professor Lanyu Shang

Announcements

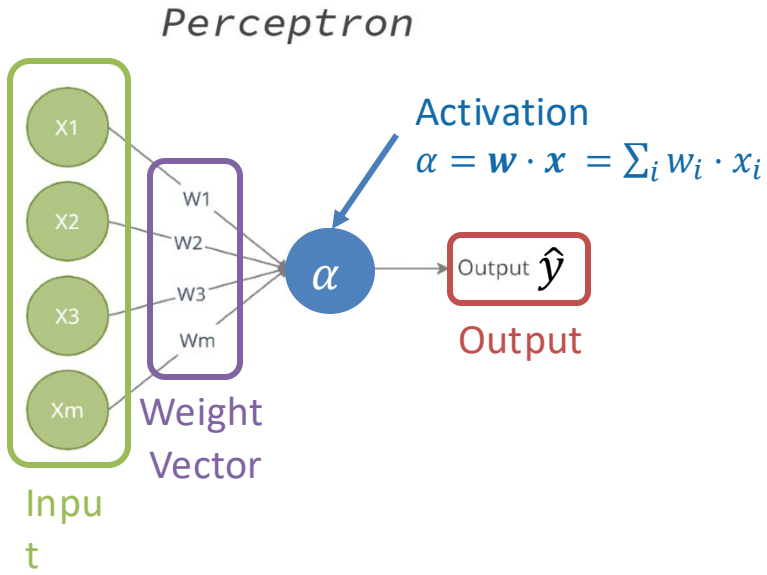
- Homework 3
  - Due tomorrow, 11/15 @5pm
- Upcoming
  - Classwork 6 – Due **Tuesday** 11/26 @5pm
  - Homework 4 – Due Friday 12/6 @5pm
  - Extra Credit Opportunities
    - Question Contribution – Due Friday 12/6 @5pm
    - One programming – Due Friday 12/13 @5pm
    - One written – Due Friday 12/13 @5pm

This Lecture

- Logistic Regression
  - Decision Rule
  - Optimization

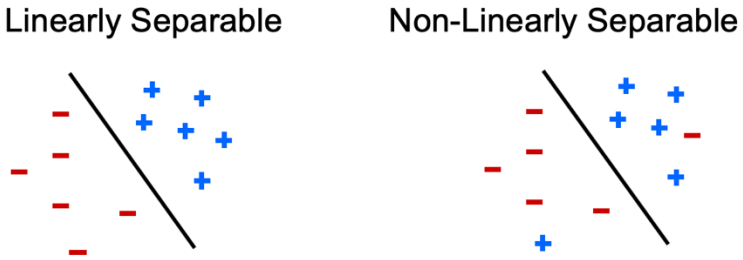
Last Lecture

- Linear Perceptron
- Learning Perceptron
  - Initialize Weights
  - Predict w/ Current Weights
  - Update Weights
    - For each **misclassification**
      - Reduce $\boldsymbol{w}_{\hat{y}}$ for misclassified class
      - Increase $\boldsymbol{w}_{y}$ for actual/true class



Last Lecture

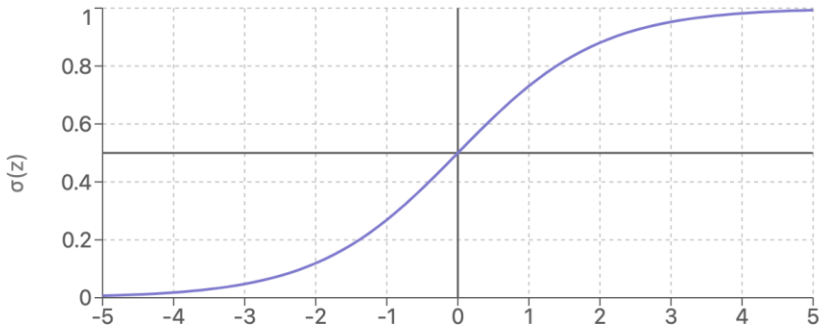
- Perceptron Properties
  - Separability
    - Linearly Separable
      - **Guarantee convergence**
      - Does not guarantee the most **generalizable** decision boundary
    - Non-linearly Separable
      - **Prevent Convergence**



How to fix it?

Logistic Regression

- Linear perceptron but with different activation and decision rule
  - Sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$
    - Scalar z => probability proportionate to magnitude



$\sigma(0) = 0.5$

$\sigma(\infty) = 1$

$\sigma(-\infty) = 0$

Logistic Regression

- z as the weighted feature in linear perceptron
  - $z = \boldsymbol{w} \cdot \boldsymbol{x}$
  - How about $\sigma(z)$?
    - If $\boldsymbol{w}$ is the weights for class $y = +1$, and if $\boldsymbol{x}$ belongs to $+1$:
      - $z = \boldsymbol{w} \cdot \boldsymbol{x} > 0$
      - $\sigma(z) = \sigma(\boldsymbol{w} \cdot \boldsymbol{x}) > 0.5$
      - $P(y^{(i)} = +1 \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}) = \sigma(\boldsymbol{w} \cdot \boldsymbol{x})$

Probability of sample i being in class y=+1.

Data sample i: ($\boldsymbol{x}^{(i)}, y^{(i)}$)

Decision Rule - Binary

- Binary Decision Rule
  - Which class (+1, -1) is more likely ($P > 0.5$)?
  - Let $\hat{y}$ be the predicted class
  - $\hat{y} = \begin{cases} +1 & \text{if } P\left(y^{(i)} = +1 \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}\right) > 0.5 \\ -1 & \text{otherwise} \end{cases}$
- How about multi-class?
  - Probabilities of a sample belonging to **each** class

Decision Rule – Multi-class

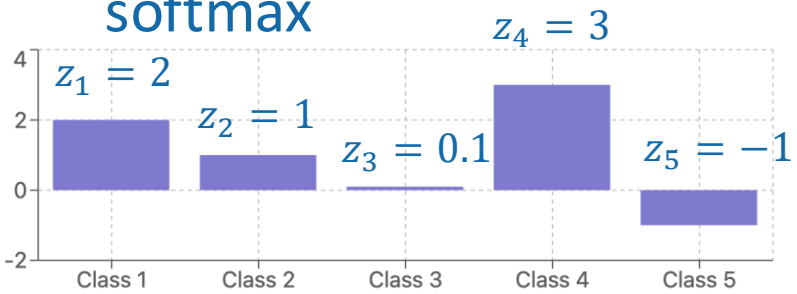
- $\boldsymbol{w}_y$ for each class $y \in Y$
- Class-specific weighted feature $z_y = \boldsymbol{w}_y \cdot \boldsymbol{x} \; \forall \; y \in Y$
- Normalized Probability
  - Softmax function: $P\left(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}\right) = \frac{e^{z_y}}{\sum_{y\prime} e^{z_{y\prime}}}$

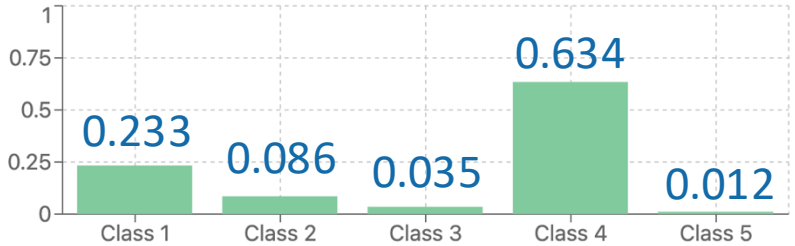
$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1}+e^{z_2}+e^{z_3}}, \frac{e^{z_2}}{e^{z_1}+e^{z_2}+e^{z_3}}, \frac{e^{z_3}}{e^{z_1}+e^{z_2}+e^{z_3}}}_{\text{softmax activations}}$$

  - Decision Rule: $\widehat{y^{(i)}} = \text{argmax}_{\text{y(i)}} P\left(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}\right)$

Decision Rule – Multi-class

- Example
  - Computing the normalized probability of class 1 using softmax





$$P(y = 1|x; w_y) = \frac{e^2}{e^2 + e^1 + e^{0.1} + e^3 + e^{-1}} \approx 0.233$$

Softmax Calculator: https://www.redcrab-software.com/en/Calculator/Softmax

Objective

- What are the optimal weights?
  - **Accuracy**: correctly classify as many sample as possible
  - **Confidence**: the highest confidence of the prediction
  - Example: 3 classes {0,1,2}, true class is $y = 1$
    - Given $P(y|x; w_y) = < 0.3, 0.4, 0.3 >$, what is $\hat{y}$?
    - The actual prob. dist. of the true class is $< 0.0, 1.0, 0.0 >$
    - How to tell if a prediction/weights is good or not across all samples?
      - Higher predicted probability on the true class, the better

Objective

- Naïve Bayes Classifier
  - $\hat{y} = \text{argmax}_{y \in Y} P(y) \prod_{x_i} P(x_i|y)$ (product of CPTs)
- Logistic Regression
  - $\boldsymbol{w}^* = \text{argmax}_{\boldsymbol{w}} \prod_i P(y^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w})$

    $= \text{argmax}_{\boldsymbol{w}} \sum_i \log P(y^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w}) \quad = \text{argmax}_{\boldsymbol{w}} ll(\boldsymbol{w})$

Best weights $\boldsymbol{w}$ from all possible values

Over **all** samples

Log-likelihood of the model's output/prediction on the **correct** class

Example

- 3 classes {0,1,2}, compute $ll(\boldsymbol{w_1})$ and $ll(\boldsymbol{w_2})$

Sample i	$P(y^{(i)}\|\boldsymbol{x}^{(i)};\boldsymbol{w})$ for $\boldsymbol{w_1}$	$P(y^{(i)}\|\boldsymbol{x}^{(i)};\boldsymbol{w})$ for $\boldsymbol{w_2}$	$y^{(i)}$ (true class)
i=1	⟨0.3,0.5,0.2⟩	⟨0.5,0.4,0.1⟩	0
i=2	⟨0.2,0.7,0.1⟩	⟨0.1,0.5,0.4⟩	2
i=3	⟨0.4,0.3,0.3⟩	⟨0.1,0.5,0.4⟩	1

$$ll(\boldsymbol{w_1}) = \sum_i \log P(y^{(i)}|\boldsymbol{x}^{(i)};\boldsymbol{w}) = \log(0.3) + \log(0.1) + \log(0.3) = -4.7$$

$$ll(\boldsymbol{w_2}) = \sum_i \log P(y^{(i)}|\boldsymbol{x}^{(i)};\boldsymbol{w}) = \log(0.5) + \log(0.4) + \log(0.5) = -2.3$$



Optimization

- "Optimize Objective"
  - **Initialization**: Start with a random initialization of $\boldsymbol{w}$
  - **Update**: Determine the step towards a neighbor (weights) that can improve the objective.
  - **Stopping**: Quit when there is no more improvement
- Demo: https://www.i-am.ai/gradient-descent.html

Optimization

- How to tell the **direction** and **magnitude** of the update?
- Derivative!
  - Slope of tangent line / rate of change
- Partial Derivative
  - Given an objective $ll(\boldsymbol{w})$ where $\boldsymbol{w} =< \mathrm{w}_1, \mathrm{w}_2 >$, the partial derivative of $ll(\boldsymbol{w})$ is $\frac{\partial ll(\boldsymbol{w})}{\partial w_i}$
    - Treat other variables $w_j \; s.t. \; j \neq i$ as **constant**
    - Tell the direction of slope for w_i at a time

Optimization

- Example
  - $f(x) = 2x^2 + 8$, what is $\frac{df(x)}{dx}$?
    - $\frac{df(x)}{dx} = 4x$
  - $g(w_1, w_2) = 5w_1^2 + 4w_2$, what are $\frac{\partial g(w_1,w_2)}{\partial w_1}$ and $\frac{\partial g(w_1,w_2)}{\partial w_2}$?
    - $\frac{\partial g(w_1,w_2)}{\partial w_1} = 10w_1$
    - $\frac{\partial g(w_1,w_2)}{\partial w_2} = 4$